

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS

COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. (BL4, BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I B Tharun Kumar / 192424356 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B Tharun Kumar

Annexure A

Pseudocode Writing Task (Algorithm Design)

Question 1

A network simulation using Cisco Packet Tracer records a total of 8000 bits received in 4 seconds. Out of 200 packets sent, 180 packets are received. An algorithm is required to compute the network throughput (in bits per second) and the packet loss percentage, validate the inputs, classify the network as Efficient or Inefficient, and display a complete performance summary.

1.1 Problem Understanding

Two independent performance metrics must be derived from the given observations:

- Throughput — the effective rate of successful data delivery over the measurement interval.
- Packet Loss Percentage — the proportion of packets that failed to reach the destination out of the total packets transmitted.

The algorithm must guard against invalid or degenerate inputs (zero time interval, zero packets sent) before performing the division operations required for these formulas, since division by zero would make the computation undefined.

1.2 Inputs, Outputs and Constraints

- Inputs: total_bits_received, time_interval (seconds), packets_sent, packets_received.
- Outputs: throughput (bps), packet_loss_percentage (%), network_status (Efficient / Inefficient), summary report.
- Constraint C1: time_interval $\neq 0$ (else throughput is undefined).
- Constraint C2: packets_sent $\neq 0$ (else packet-loss ratio is undefined).

- Constraint C3: Efficient network $\Leftrightarrow$ throughput > 1500 bps AND packet_loss_percentage $< 10\%$.

1.3 Governing Formulas

Throughput (bits per second):

$$\text{Throughput} = \text{Total_Bits_Received} / \text{Time_Interval}$$

Packet Loss Percentage:

$$\text{Packet_Loss (\%)} = ((\text{Packets_Sent} - \text{Packets_Received}) / \text{Packets_Sent}) \times 100$$

Classification Rule:

Status = Efficient, if (Throughput > 1500) AND (Packet_Loss < 10); else Status = Inefficient

1.4 Algorithm Design Strategy (Decomposition)

- Sub-problem A — Input acquisition and validation (guard clauses for C1 and C2).
- Sub-problem B — Metric computation (apply the two formulas above).
- Sub-problem C — Classification logic (apply the compound Boolean rule).
- Sub-problem D — Result presentation (structured summary output).

1.5 Pseudocode

```
BEGIN NetworkThroughputAndLossAnalysis
  DECLARE total_bits_received, time_interval AS REAL
  DECLARE packets_sent, packets_received AS INTEGER
  DECLARE throughput, packet_loss AS REAL
  DECLARE network_status AS STRING

  INPUT total_bits_received, time_interval
  INPUT packets_sent, packets_received
```

```

// ---- Validation Module ----
IF (time_interval = 0) THEN
    PRINT "Error: Time interval cannot be zero."
    EXIT ALGORITHM
END IF

IF (packets_sent = 0) THEN
    PRINT "Error: Packets sent cannot be zero."
    EXIT ALGORITHM
END IF

// ---- Computation Module ----
throughput = total_bits_received / time_interval
packet_loss = ((packets_sent - packets_received) / packets_sent) * 100

// ---- Classification Module ----
IF (throughput > 1500) AND (packet_loss < 10) THEN
    network_status = "Efficient"
ELSE
    network_status = "Inefficient"
END IF

// ---- Output Module ----
PRINT "----- Network Performance Summary -----"
PRINT "Throughput      : ", throughput, " bps"
PRINT "Packet Loss     : ", packet_loss, " %"
PRINT "Network Status  : ", network_status
END NetworkThroughputAndLossAnalysis

```

1.6 Trace / Verification with Given Values

Throughput = $8000 / 4 = 2000$ bps

Packet_Loss = $((200 - 180) / 200) \times 100 = (20 / 200) \times 100 = 10\%$

Classification check: $\text{throughput} (2000) > 1500 \rightarrow \text{TRUE}$, but $\text{packet_loss} (10\%) < 10\% \rightarrow \text{FALSE}$ (10 is not strictly less than 10). Since the compound AND condition requires both sub-conditions to hold, the network is classified as Inefficient despite the healthy throughput — illustrating why the algorithm must evaluate the full Boolean expression rather than a single metric in isolation.

1.7 Explanation of Design Choices

The validation module is placed before any arithmetic operation so that a zero denominator can never reach the division statements — this is a defensive-programming pattern that keeps the algorithm robust against malformed simulation logs. The two metrics are computed independently because they capture orthogonal aspects of performance: throughput reflects volume/speed of delivery, while packet loss reflects reliability of delivery; a network can score well on one and poorly on the other, which is why the classification rule combines both with a logical AND rather than evaluating them separately. Modularizing the algorithm into validation, computation, classification and output blocks keeps each concern independently testable and reusable for other datasets captured from Packet Tracer.

Question 2

An organization with multiple branch links needs continuous monitoring of bandwidth utilization so that overloaded links can be detected and traffic redirected to a backup link before congestion affects service quality.

2.1 Problem Understanding

The problem is a continuous (real-time / periodic) monitoring problem rather than a single-shot calculation. It requires: a data structure to hold the state of every link, a repeating collection cycle, a threshold-based decision rule per link, and a corrective action (failover) when the rule is triggered.

2.2 Inputs, Outputs and Constraints

- Inputs: link_capacity[i], link_current_usage[i] for every link i in the network, sampling_interval, threshold = 80%.
- Outputs: utilization[i] for every link, an overload alert and backup-switch action whenever utilization[i] exceeds the threshold.
- Constraint C1: link_capacity[i] $\neq$ 0 for all i (utilization is undefined otherwise).
- Constraint C2: Overload(i) $\Leftrightarrow$ utilization[i] > 80%.
- Constraint C3: monitoring must repeat indefinitely (or for the duration of operation) at fixed sampling_interval, not just once.

2.3 Governing Formula

Bandwidth Utilization of Link i:

$$\text{Utilization[i] (\%)} = (\text{Current_Usage[i]} / \text{Link_Capacity[i]}) \times 100$$

Overload Condition:

$$\text{Overload[i]} = \text{TRUE, if Utilization[i]} > 80; \text{ else Overload[i]} = \text{FALSE}$$

2.4 Problem Decomposition

- Sub-problem A — Data structure design: parallel arrays/lists for link identifiers, capacities and current usage readings.
- Sub-problem B — Periodic sampling loop: an infinite/timed loop that re-reads usage values at every `sampling_interval`.
- Sub-problem C — Per-link evaluation: apply the utilization formula and overload condition to each element of the list.
- Sub-problem D — Failover action: redirect traffic of an overloaded link to its associated backup link and raise an alert.

2.5 Pseudocode

```
BEGIN BandwidthMonitoring

  DECLARE link_id[1..N] AS STRING
  DECLARE link_capacity[1..N] AS REAL
  DECLARE backup_link[1..N] AS STRING
  DECLARE utilization[1..N] AS REAL
  DECLARE THRESHOLD = 80

  INITIALIZE link_id, link_capacity, backup_link FROM
  network_configuration

  LOOP FOREVER // continuous monitoring cycle
    WAIT sampling_interval
    FOR i = 1 TO N DO
      current_usage[i] = READ_CURRENT_USAGE(link_id[i])
      IF link_capacity[i] = 0 THEN
        PRINT "Warning: capacity undefined for ", link_id[i]
        CONTINUE
      END IF
      utilization[i] = (current_usage[i] / link_capacity[i]) * 100

      IF utilization[i] > THRESHOLD THEN
```

```

        PRINT "ALERT: ", link_id[i], " overloaded at ",
        utilization[i], "%"
        SWITCH_TRAFFIC(link_id[i], backup_link[i])
        PRINT "Traffic redirected to backup link: ", backup_link[i]
    ELSE
        PRINT link_id[i], " operating normally at ", utilization[i],
        "%"
    END IF
END FOR
END LOOP
END BandwidthMonitoring

```

2.6 Explanation — How the Algorithm Supports Bandwidth Management

Representing every link's capacity and live usage in indexed arrays allows the same evaluation logic (a single FOR loop) to scale to any number of branch links without rewriting the algorithm — this satisfies the decomposition constraint of treating monitoring as a repeatable, data-driven process rather than a hard-coded check per link. The outer infinite loop combined with a WAIT step turns a single measurement into an ongoing surveillance process, which is essential because bandwidth demand fluctuates continuously throughout the day; a one-time check would miss overload conditions that develop later. The 80% threshold acts as an early-warning boundary set below full saturation (100%), giving the network administrator a safety margin to react — this is the core constraint the whole decision block is built around. When the condition is triggered, immediately invoking SWITCH_TRAFFIC prevents the link from reaching hard saturation, which is the direct mechanism by which the algorithm prevents congestion rather than merely reporting it after the fact. Because each link also carries its own designated backup_link, the failover is targeted and local, avoiding unnecessary disruption to links that are not overloaded.

Question 3

Six branch offices are connected to headquarters through multiple links whose bandwidth, delay and utilization vary throughout the day. An algorithm is required to continuously evaluate overall link quality (not merely physical distance), select the best path per branch, detect overload or failure, switch to an alternate path automatically, and alert the administrator.

3.1 Problem Understanding

This is an adaptive, multi-criteria routing problem. Unlike Question 2, a single threshold on one metric is insufficient — three metrics (bandwidth, delay, utilization) must be combined into one composite score so that different paths can be ranked and compared fairly. The algorithm must also run continuously, since link quality is time-varying, and must include fault-tolerance logic for both gradual degradation (overload) and abrupt failure.

3.2 Inputs, Outputs and Constraints

- Inputs, per link j serving branch b : bandwidth[j] (Mbps), delay[j] (ms), utilization[j] (%), link_state[j] (UP/DOWN).
- Outputs: quality_score[j] for every candidate link, best_path per branch, overload/failure alerts, automatic path switch-over.
- Constraint C1: only links with link_state = UP are eligible for selection.
- Constraint C2: higher bandwidth should increase the score; higher delay and higher utilization should decrease the score (they are inversely related to quality).
- Constraint C3: Overload(j) $\Leftrightarrow$ utilization[j] > 80% (consistent with Question 2's safety margin).
- Constraint C4: Failure(j) $\Leftrightarrow$ link_state[j] = DOWN, which must trigger immediate re-selection, not merely an alert.

- Constraint C5: the monitoring/selection cycle must repeat at fixed intervals to adapt to changing conditions.

3.3 Governing Formula — Composite Link Quality Score

Because bandwidth is a benefit factor while delay and utilization are cost factors, the metrics are first normalized to a common 0–1 range and combined using a weighted-sum multi-criteria formula:

$$\text{Normalized Bandwidth: } NB[j] = \text{Bandwidth}[j] / \text{Bandwidth_max}$$

$$\text{Normalized Delay (inverted): } ND[j] = 1 - (\text{Delay}[j] / \text{Delay_max})$$

$$\text{Normalized Utilization (inverted): } NU[j] = 1 - (\text{Utilization}[j] / 100)$$

Composite Quality Score:

$$\text{Score}[j] = (W1 \times NB[j]) + (W2 \times ND[j]) + (W3 \times NU[j]), \text{ where } W1 + W2 + W3 = 1$$

The weights $W1$, $W2$, $W3$ let the administrator emphasize whichever factor matters most operationally (e.g. a higher $W2$ for delay-sensitive video traffic); a typical balanced starting point is $W1 = W2 = W3 = 0.33$. The path chosen for each branch is:

$$\text{Best_Path(branch)} = \text{argmax}(\text{Score}[j]), \text{ for all } j \text{ with link_state}[j] = \text{UP}$$

3.4 Problem Decomposition

- Sub-problem A — Metric collection: gather bandwidth, delay and utilization for every candidate link of every branch at each cycle.
- Sub-problem B — Normalization: rescale each raw metric onto a comparable 0–1 scale using the maxima observed across candidate links.
- Sub-problem C — Scoring: combine normalized metrics into one composite score per link using the weighted-sum formula.
- Sub-problem D — Selection: choose the UP link with the maximum score as the active path for each branch.

- Sub-problem E — Fault handling: continuously watch for overload or DOWN state and force an immediate re-selection plus an alert.
- Sub-problem F — Repetition: wrap A–E inside a timed loop so the best path adapts as conditions change through the day.

3.5 Pseudocode

```

BEGIN AdaptivePathSelection

  DECLARE branch[1..6] AS STRING
  DECLARE link_id[1..M] AS STRING
  DECLARE bandwidth[1..M], delay[1..M], utilization[1..M] AS REAL
  DECLARE link_state[1..M] AS STRING    // "UP" or "DOWN"
  DECLARE score[1..M] AS REAL
  DECLARE W1 = 0.33, W2 = 0.33, W3 = 0.34
  DECLARE THRESHOLD_UTIL = 80

  LOOP FOREVER    // periodic adaptive monitoring
    WAIT monitoring_interval

    FOR each branch b DO
      candidate_links = GET_LINKS_FOR(b)
      bw_max = MAX(bandwidth OF candidate_links)
      dl_max = MAX(delay OF candidate_links)

      FOR each link j IN candidate_links DO
        READ bandwidth[j], delay[j], utilization[j], link_state[j]

        IF link_state[j] = "DOWN" THEN
          PRINT "ALERT: Link ", link_id[j], " has FAILED"
          score[j] = -INFINITY    // remove from contention
          CONTINUE
        END IF
      
```

```

    IF utilization[j] > THRESHOLD_UTIL THEN
        PRINT "ALERT: Link ", link_id[j], " overloaded at ",
            utilization[j], "%"
    END IF

    NB = bandwidth[j] / bw_max
    ND = 1 - (delay[j] / dl_max)
    NU = 1 - (utilization[j] / 100)
    score[j] = (W1 * NB) + (W2 * ND) + (W3 * NU)
END FOR

best_link = LINK WITH MAX(score) AMONG candidate_links
IF best_link ≠ current_active_path[b] THEN
    SWITCH_ACTIVE_PATH(b, best_link)
    PRINT "Path for ", b, " switched to ", best_link, score = ",
        score[best_link]
    PRINT "Administrator alert: route change on branch ", b
ELSE
    PRINT b, " remains on optimal path ", best_link
END IF
END FOR
END LOOP
END AdaptivePathSelection

```

3.6 Worked Illustration

Suppose branch B1 has two candidate links: Link-1 (bandwidth = 100 Mbps, delay = 40 ms, utilization = 30%) and Link-2 (bandwidth = 60 Mbps, delay = 10 ms, utilization = 85%). Here $bw_max = 100$, $dl_max = 40$.

Link-1 $\rightarrow NB = 100/100 = 1.00$, $ND = 1 - 40/40 = 0.00$, $NU = 1 - 0.30 = 0.70 \rightarrow \text{Score} = 0.33(1.00) + 0.33(0.00) + 0.34(0.70) = 0.568$

Link-2 $\rightarrow$ NB = $60/100 = 0.60$, ND = $1 - 10/40 = 0.75$, NU = $1 - 0.85 = 0.15$ (also flagged OVERLOADED since $85\% > 80\%$) $\rightarrow$ Score = $0.33(0.60) + 0.33(0.75) + 0.34(0.15) = 0.497$

Even though Link-2 has lower delay, its very high utilization pulls its composite score below Link-1's, so Link-1 is selected as the active path — this demonstrates why a composite, weighted formula is more appropriate than choosing a path on a single metric such as shortest delay.

3.7 Explanation — Reliability, Routing Efficiency and Fault Tolerance

Reliability is improved because the algorithm treats a DOWN link as permanently disqualified for that cycle (score forced to $-\infty$) and immediately re-runs the argmax selection, so headquarters-to-branch connectivity is restored automatically without waiting for manual intervention. Routing efficiency is improved because the decision is no longer based on a single static property (such as physical distance or hop count) but on a live composite score that reflects the actual carrying capacity, latency and congestion state of each link at the moment of evaluation — this lets the algorithm continually route traffic over whichever path currently offers the best real performance, not just the theoretically shortest one. Fault tolerance is built in at two levels: a soft level (utilization exceeding 80% raises an alert while the link is still usable, giving early warning consistent with the overload rule from Question 2) and a hard level (link_state = DOWN forces exclusion and automatic failover). Wrapping the entire evaluation in a periodic LOOP FOREVER means the algorithm keeps adapting as bandwidth, delay and utilization drift throughout the day, so the "best path" is recomputed rather than fixed — this is what allows the design to respond to changing network conditions instead of only reacting to conditions that existed at deployment time.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient